

ОТЧЁТ

по решениям заданий демонстрационного экзамена

Специальность 09.02.07
КИМ 09.02.07-5-2027

Выполнил(а): _____
Дата: 07 октября 2026 г.

Краткое резюме

Подготовлены решения для семи заданий: ER-модель, SQLite-база, SQL-расчёт себестоимости, приложение авторизации, API заметок и тесты, документация, очищенные данные и анализ месячной выручки. Исходные экзаменационные файлы не изменялись.

Задание	Результат
1	ER-диаграмма в ЗНФ.
2	Схема БД, импорт JSON, проверка ссылок.
3	Расчёт себестоимости заказа по нормам и ценам.
4	Авторизация, капча-пазл, роли, блокировка, панель администратора.
5	API GET /notes, Postman-коллекция и автоматические тесты.
6	Документация в шаблоне из приложения.
7	Очистка 30 транзакций, годовой анализ и график со средним.

Задание 1. Проектирование ER-диаграммы

Модель описывает изготовление изделий и заказы покупателей. Материалы и технологические операции объединены в `resources` и различаются `resource_type`. Спецификация связывает товары с ресурсами и хранит норму расхода на изделие.

Таблица	Ключевые поля / назначение
customers — Заказчики	PK customer_id; name, INN, address, phone, party_type
customer_orders — Заказы покупателей	PK order_id; FK customer_id; order_date, status
order_items — Позиции заказа	PK order_item_id; FK order_id/product_id; quantity; снимок цены и скидка на единицу
products — Товары	PK product_id; source_product_id; name, category, is_placeholder
resources — Материалы и операции	PK resource_id; source_resource_id; name, resource_type
product_specification — Спецификация	PK specification_id; FK product_id/resource_id; quantity_per_product
prices — Цены	PK price_id; FK product_id ИЛИ resource_id; unit_price
discounts — Скидки	PK discount_id; FK product_id ИЛИ resource_id; discount_percent
production_orders — Заказы на производство	PK production_order_id; order_number, launch_date, department
production_order_items — Строки выпуска	PK production_order_item_id; FK production_order_id/product_id; quantity, unit

Связи и ЗНФ

Заказчик 1:N заказы; заказ 1:N позиции; товар 1:N позиции. Товары и ресурсы связаны M:N через `product_specification`. Производственный заказ 1:N строк выпуска. Цены и скидки имеют ровно одну целевую ссылку — на товар либо на ресурс. Снимок цены и скидка в позиции заказа отделены от текущего прайс-листа.

В источниках обнаружены повторные ID товара 887, ресурса 456, строк заказа 31/81 и скидки 15. Они сохранены с surrogate-ключами. Для отсутствующих `product_id` 398, 881 и 942 созданы помеченные заглушки. ER-диаграмма: `task1_er/ER-диаграмма.pdf`.

Задание 2. Создание базы данных

Созданы таблицы `customers`, `customer_orders`, `order_items`, `products`, `resources`, `product_specification`, `prices`, `discounts`, `production_orders` и `production_order_items`. Скрипт `seed.py` импортирует JSON и проверяет внешние ключи: нарушений нет.

Таблица	Число записей
customers	6
products	132 (включая 3 заглушки)
resources	103
product_specification	74
customer_orders	70
order_items	70
prices	60
discounts	51

python3 solutions/task2_database/seed.py

Задание 3. Запрос себестоимости

Себестоимость компонента заказа = количество заказанных изделий × норма расхода × цена ресурса. Материалы и операции учитываются одинаково. При отсутствии цены итог помечается как неполный, а не маскируется нулём.

```
WITH components AS (
  SELECT o.order_id, oi.quantity AS ordered_quantity,
         ps.specification_id, ps.quantity_per_product, p.price_id, p.unit_price
  FROM customer_orders o JOIN order_items oi ON oi.order_id=o.order_id
  LEFT JOIN product_specification ps ON ps.product_id=oi.product_id
  LEFT JOIN prices p ON p.resource_id=ps.resource_id AND p.product_id IS NULL
  WHERE o.order_id=:order_id
)
SELECT order_id,
       CASE WHEN COUNT(specification_id)=0 OR COUNT(price_id)<>COUNT(specification_id)
            THEN NULL ELSE ROUND(SUM(ordered_quantity*quantity_per_product*unit_price),2)
       END AS total_manufacturing_cost
FROM components GROUP BY order_id;
```

Ресурс	На заказ, руб.
Евровинт 6,5х5	10,37
Мебельная деталь 500х800	150,00
Мебельная деталь 600х800	137,00
Опора	1165,75
Столешница круглая	2485,00
Распил ДСП, МДФ и листового материала	900,00
Сборка модулей	2800,00
Упаковка	3800,00
Итого	11448,12

Заказ № 1 — стол «Самобранка», 2 шт.; итог 11448,12 руб. Это совпадает с приложенным расчётом стоимости.

Задание 4. Модуль авторизации

Приложение написано на Python и SQLite. Есть роли «Администратор» и «Пользователь», обязательные поля входа, пазл-капча 3×3 на основе приложенных рисунков, серверная проверка решения и блокировка после трёх последовательных ошибок капчи или пароля. Пароли хранятся как PBKDF2-HMAC-SHA256 с солью.

Панель /admin/users позволяет добавлять учётные записи, проверять уникальность логина, менять логин, роль и пароль, блокировать и разблокировать пользователя. Демонстрационные входы: admin / Admin123! и user / User123!.

python3 solutions/task4_auth_api/app.py # http://127.0.0.1:8000

Задание 5. API и тестирование

GET /notes (алиас GET /api/notes) возвращает JSON-массив заметок. title_user = title + « - » + login; content передаётся без изменений; created_at преобразуется в ДД.ММ.ГГГГ. Необязательные параметры: limit (1–1000) и user_id (>0).

```
[
  {
    "id": 5,
    "title_user": "Итоги - admin",
    "content": "Сохранить результаты тестирования.",
    "formatted_date": "05.10.2026"
  }
]
```

Ситуация	Код	Ответ
Штатно; нет заметок	200	JSON-массив; при отсутствии данных []
Некорректный параметр	400	JSON с error
Сбой БД	500	JSON с описанием ошибки
Метод не GET	405	JSON и Allow: GET

Коллекция Postman приложена. Автотесты покрывают 200, 400, 500, JSON-формат, пустой массив, 405, блокировку и администрирование. Пройдено 7 тестов.

Задание 6. Документация API

По шаблону из приложения описаны базовый URL, GET /notes и алиас, параметры, формат данных, HTTP-статусы и примеры запросов/ответов 200, 400, 500. Документ: task6_docs/API_документация.docx.

Задание 7. Очистка и анализ данных

Обработано 30 транзакций за 2026 год; полных дубликатов нет, строки не удалялись. Нормализованы ID, названия, категории и имена менеджеров; текстовые количества/цены преобразованы в числа; исправлена опечатка Dison → Dyson. В журнале — 47 исправлений в 27 строках.

Выручка по месяцам

Месяц	Выручка, руб.	Доля, %
Январь	413985	5,34
Февраль	1278414	16,48
Март	1208502	15,57
Апрель	526072	6,78
Май	279840	3,61
Июнь	598556	7,71
Июль	234172	3,02
Август	332508	4,29
Сентябрь	127949	1,65
Октябрь	716678	9,24
Ноябрь	1794540	23,13
Декабрь	248464	3,20
Итого	7759680	100,00

Показатель	Результат
Средняя месячная выручка	646640 руб.
Максимум	Ноябрь — 1794540 руб.
Минимум	Сентябрь — 127949 руб.

Показатель	Результат
Размах	1666591 руб.
Линия среднего на графике	646640 руб.

Линейный график содержит месяцы в хронологическом порядке и горизонтальную серию среднего значения. Ряд волатилен; устойчивого непрерывного роста нет.

Ограничения и оговорки

В TXN-0001 цена утюга Philips равна 897270 руб./шт. Это сильный ценовой выброс, но в приложениях нет подтверждённой цены или правила замены; исходное число сохранено и помечено для сверки. Оно формирует ноябрьский пик.

В загруженных материалах нет Analytics_autotest и отдельных формулировок вопросов 7.2–7.9. Официальный контрольный код 7.1 надо получить запуском экзаменационного автотестера; SNA-256 файла не является контрольной суммой приложения.

Файлы решения

Задания	Основные файлы
1	task1_er/ER_диаграмма.pdf; ER-модель.mmd
2	task2_database/schema.sql; seed.py; manufacturing.sqlite3
3	task3_query/order_cost.sql; пример_расчёта.md
4–5	task4_auth_api/app.py; tests; task5_api/Exam_API.postman_collection.json
6	task6_docs/API_документация.docx
7	task7_analysis/clean_dataset_2026.xlsx; analysis_2026.xlsx; ответы_к_заданию_7.md

Подробные команды запуска приведены в solutions/README_RU.md.